

Documento de Arquitetura

Lucas Tuon de Matos

Link do Repositório:

<https://github.com/LucasTuon/TelaDeAnaliseDeUsuarios>

1 - Arquitetura do Sistema

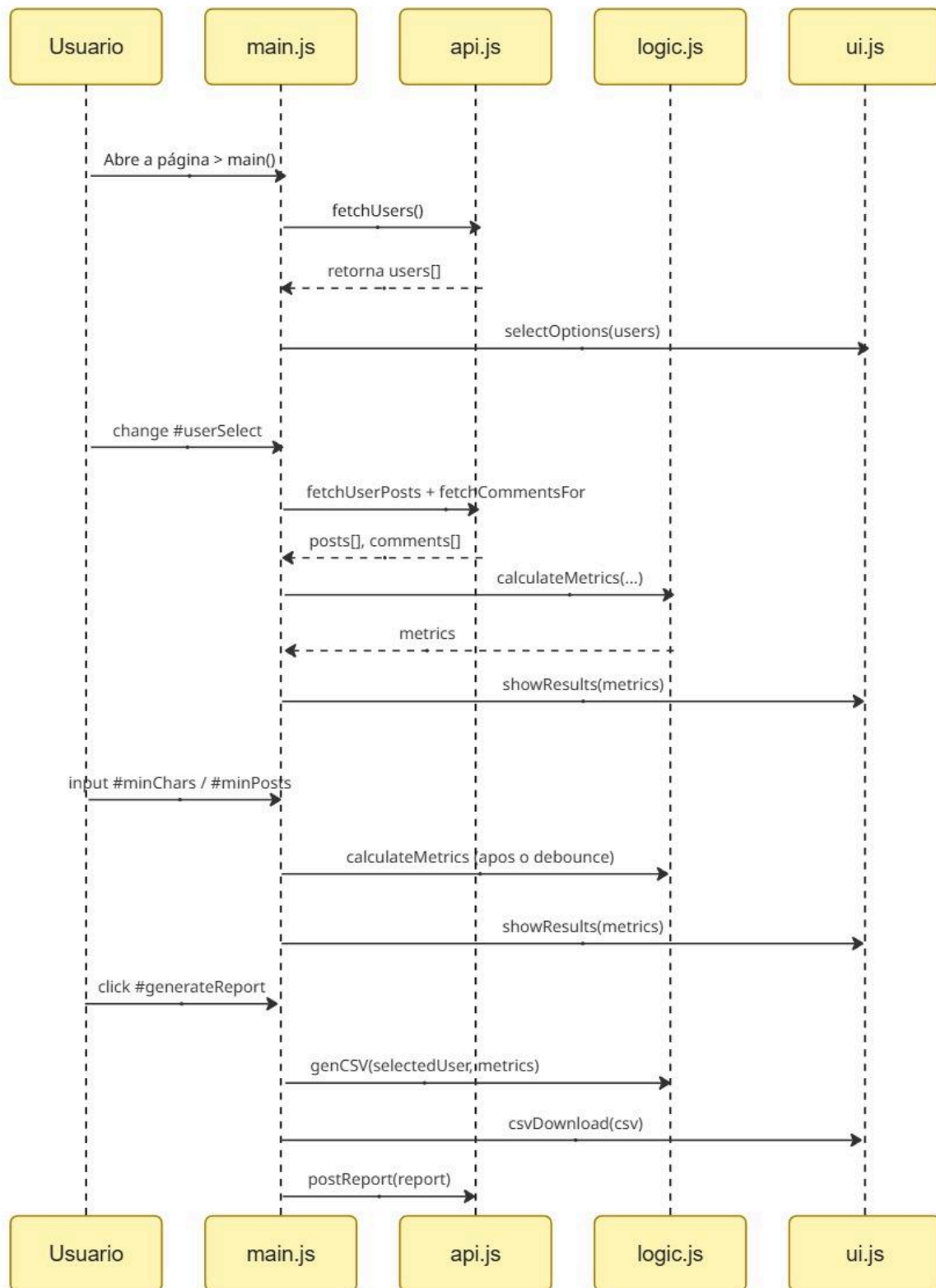
O sistema foi dividido em quatro módulos de JavaScript:

- **api.js:** Responsável por buscar dados brutos da API JSONPlaceholder (GET) e simular o envio do relatório (POST).
- **logic.js:** Responsável por transformar os dados brutos em métricas (quantidade de posts, médias, status) e montar o conteúdo do relatório CSV.
- **ui.js:** Responsável pela manipulação do DOM (adicionar os usuários no Select, exibir os resultados na tela e disparar o download do CSV).
- **main.js:** Ponto de entrada da aplicação. Importa as funções dos outros três módulos, gerencia os estados e registra os event listeners.

Essa organização facilita a manutenção e entendimento do código, pois segue uma arquitetura modular que pode ser entendida parte por parte.

2 - Fluxo de Eventos

A aplicação segue o modelo orientado a eventos, reagindo aos acontecimentos em tempo real. Segue o Diagrama de Sequência dos eventos:



Ao abrir a página, *main()* é chamada, busca os usuários via *fetchUsers()* e completa o `<select>` com *selectOptions()*. O evento *change* no `#userSelect` dispara *userData(userId)*, que busca posts e comentários do usuário, calcula as métricas e exibe o resultado. A alteração dos filtros em `#minChars` e `#minPosts` dispara *recalculateMetricsDebounce()*, que recalcula as métricas com base nos dados já carregados (sem novo *fetch*) e atualiza a tela. Um *debounce* de 500ms evita recálculos excessivos durante a digitação. O evento *click* em `#generateReport` gera o CSV, dispara o download e simula o envio POST para `/reports`.

3 - Linguagens, Bibliotecas e Ferramentas

- **Linguagem:** JavaScript (ES6+)
- **Módulos:** ES Modules nativos (import/export)
- **API Consumida:** JSONPlaceholder - <https://jsonplaceholder.typicode.com/>
- **Recursos Nativos:**
 - *fetch* - Requisições HTTP (GET e POST)
 - *Promise.all* - Paralelização de requisições dos comentários
 - *Blob* e *URL.createObjectURL* - Geração e download do CSV
 - *setTimeout* e *clearTimeout* - Implementação do Debounce
- **Hospedagem:** GitHub Pages (necessário porque os ES Modules exigem servidor HTTP)
- **Bibliotecas:** Nenhuma biblioteca externa foi utilizada.

4 - Decisões Técnicas

Cache de dados em memória: os posts e comentários buscados na Task 2 são armazenados em variáveis no *main.js* (*postsData*, *commentsData*), evitando novas requisições à API a cada alteração de filtro. Os filtros da Task 3 recalculam sobre esses dados já carregados, sem novo *fetch*, reduzindo chamadas desnecessárias.

Debounce nos filtros: aplicado um atraso de 500ms nos eventos de *input* (*minChars*, *minPosts*) via função *debounceEvent*, evitando recálculos a cada tecla digitada. O recálculo só ocorre quando o usuário para de digitar por 300ms.

Tratamento de erro com *try/catch* + *response.ok*: o *fetch* não lança erro automaticamente para status HTTP como 404 ou 500. A Promise resolve apenas com *response.ok* igual a *false*. Por isso, cada função de busca em *api.js* verifica *response.ok* e lança erro manualmente (*throw new Error*), permitindo que o *try/catch* em *main.js* capture falhas em toda a aplicação, tanto nas requisições GET quanto no POST de simulação de envio.

Estrutura modular sem bundler: ES Modules nativos (*import/export*) em vez de um único arquivo, separando responsabilidades entre *api.js*, *logic.js* e *ui.js*. Essa decisão exige que a aplicação seja HTTP (não funciona abrindo o HTML diretamente do disco), por isso o projeto foi publicado no GitHub Pages.

Geração de CSV sem biblioteca externa: o relatório é montado manualmente como uma string (cabeçalho + linha de dados separados por vírgula) e baixado via *Blob* e *URL.createObjectURL*.

Escopo do relatório (Task 4): o relatório reflete os dados do usuário atualmente selecionado, já com os filtros aplicados (os mesmos valores exibidos na tela no momento do clique em "Gerar Relatório"), garantindo coerência entre o que o usuário vê e o que é exportado.

Requisições em paralelo: os comentários de cada post são buscados simultaneamente com *Promise.all*, em vez de sequencialmente com *await* dentro de um loop. Como as requisições são independentes entre si, essa abordagem reduz significativamente o tempo total de espera.

5 - Problemas e Soluções

Timing entre função assíncrona e sua dependente: funções que dependiam do resultado de um *fetch* foram chamadas fora da função *async* que buscava os dados, antes da resposta chegar. Isso resultava em variáveis *undefined* quando fosse usado. **Solução:** mover a chamada da função dependente para dentro da função *async*, executando-a apenas após o *await* da resposta.

Interpolação de string sem Template Literals: as URLs dinâmicas foram inicialmente escritas com aspas simples em vez de crase, fazendo com que a variável interpolada fosse enviada como texto literal em vez do valor e a API retornava vazio. **Solução:** usar Template Literals sempre que houvesse interpolação de variável na URL.

Divisão por zero (NaN/Infinity): os resultados das métricas ficaram como NaN e Infinity quando o filtro de caracteres estava muito alto e fazia a quantidade de posts filtrados chegar a zero. **Solução:** adicionar verificação condicional antes de cada divisão, retornando zero quando a quantidade de posts for zero.

Erro HTTP não capturado pelo try/catch: o *fetch* não rejeita a Promise para respostas HTTP de erro (como 404), apenas para falhas de rede e isso fazia com que o erro de POST para */reports*, (endpoint inexistente na API) passasse despercebido pelo tratamento de erro. **Solução:** verificar manualmente *response.ok* após cada *fetch* e lançar um erro quando a resposta não for bem-sucedida.

Chamada de relatório sem usuário selecionado: era possível clicar em "Gerar Relatório" antes de selecionar qualquer usuário e isso causava um erro ao tentar acessar propriedades de um valor *undefined*. **Solução:** validar a existência do usuário selecionado no início do listener do botão, interrompendo a execução com uma mensagem de erro caso nenhum usuário tenha sido escolhido.

Retorno perdido ao combinar debounce com cálculo: a função de debounce foi inicialmente aplicada diretamente sobre a função de cálculo de métricas, que possui retorno. Como o debounce agenda a execução para um momento posterior, o valor de retorno não estava mais disponível no momento da chamada, que resultava num *undefined*. **Solução:** aplicar o debounce sobre uma outra função, que calcula e já exibe o resultado internamente, em vez de depender do valor de retorno.

Chamada duplicada de renderização (showResults): a função *showResults* estava sendo chamada tanto dentro da função *recalculateMetrics* quanto separadamente no listener do evento de input. Como o debounce atrasa a execução de *recalculateMetrics*, a chamada externa a *showResults* rodava antes do cálculo terminar, tentando desestruturar um valor ainda *undefined* e lançando erro.

Solução: manter a chamada a *showResults* apenas dentro de *recalculateMetrics*, garantindo que ela só execute depois que *metrics* já tiver sido calculado.

6 - Referências

Eventos em JS -

<https://demenezes.dev/posts/guia-eventos-em-javascript/>

Tutorial DropDown List -

<https://www.youtube.com/watch?v=3kL-P9FYjbc>

createElement e appendChild -

https://www.youtube.com/watch?v=ucfNgEI_Vcw

promise e async/await em loops -

<https://dev.to/oieduardorabelo/javascript-armadilhas-do-async-await-em-loops-4j36>

Estrutura Modular de Código -

<https://dev.to/dougsorce/como-eu-estruturo-meus-projetos-vanilla-js-3dl6>

Função Debounce -

<https://www.youtube.com/watch?v=OyTPNNly3pc>

Tratamento de Erro com response.ok -

<https://dev.to/dionarodrigues/fetch-api-do-you-really-know-how-to-handle-errors-2gj0>